CSA1708-ARTIFICIAL INTELLIGENCE

LAB EXPERIMENTS

Program 1: Python program to solve 8.Puzzle Problem.

Aim:

To write a Python program to solve the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm and find the sequence of moves from the initial state to the goal state.

Algorithm:

1. Start the program.
2. Enter the initial and goal states.
3. Use BFS with a queue to store puzzle states.
4. Find the blank space (0) and generate possible moves.
5. Add unvisited states to the queue.
6. Repeat until the goal state is reached.
7. Display the solution path and number of moves.
8. Stop.

lab.py - C:/Users/navit/lab.py (3.4.3)

File Edit Format Run Options Window Help

```
from collections import deque

# Function to print the puzzle
def print_puzzle(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()


# Possible movements of blank space
moves = {
    0: [1, 3],
    1: [0, 2, 4],
    2: [1, 5],
    3: [0, 4, 6],
    4: [1, 3, 5, 7],
    5: [2, 4, 8],
    6: [3, 7],
    7: [4, 6, 8],
    8: [5, 7]
}


# BFS function
def solve_8_puzzle(start, goal):
    queue = deque([(start, [])])
    visited = set()
    visited.add(tuple(start))

    while queue:
        state, path = queue.popleft()

        if state == goal:
            return path + [state]

        blank = state.index(0)

        for new_blank in moves[blank]:
            new_state = state.copy()

            # Swap blank with adjacent tile
            new_state[blank], new_state[new_blank] = \
                new_state[new_blank], new_state[blank]

            if tuple(new_state) not in visited:
                visited.add(tuple(new_state))
                queue.append((new_state, path + [state]))
```

Python 3.4.3 Shell - C:/Users/navit/lab 1.py (3.4.3)

File Edit Shell Debug Options Window Help

```
Python 3.4.3 (v3.4.3:9b73f1c3e601, Feb 24 2015, 22:44:40) [MSC v.1600 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> ========== RESTART ==========
>>>
Enter the initial state (use 0 for blank):
1 2 3 4 0 6 7 5 8

Solution found!
Number of moves: 2
Step 0
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

Step 1
[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

Step 2
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

>>>
```

Result:

Thus, the **8-Puzzle problem was successfully solved using BFS in Python**, and the solution path was obtained.